

A. Divide and Conquer

1 second, 256 megabytes

You are given two positive integers x and y .

You are allowed to perform the following operation any number of times (possibly zero):

- Choose any positive integer z such that z divides x ;
- Set $x := \frac{x}{z}$.

Determine whether you can make x exactly equal to y using this operation.

Input

The first line of the input contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of each test case follows.

The only line of each test case contains two space-separated integers x and y ($1 \leq x, y \leq 100$).

Output

For each test case, print "YES" if you can make x exactly equal to y and "NO" otherwise.

You can output "YES" and "NO" in any case (for example, strings "yEs", "yes" and "Yes" will be recognized as a positive response).

input
3
12 2
6 7
99 79
output
YES
NO
NO

For the first test case,

- Choosing $z = 2$ makes $x = \frac{12}{2} = 6$
- Choosing $z = 3$ makes $x = \frac{6}{3} = 2$

Thus, we can make $x = y$. Hence, output YES.

For the second test case, it can be shown that it is impossible to make x equal to y .

B. Good times Good times

1 second, 256 megabytes

An integer n is said to be *good* if it contains **at most** two distinct digits in its decimal representation. For example, the integers 3, 8588, 67 are good, whereas the integers 123, 9447 are not.

You are given an integer x ($1 \leq x < 10^8$), which is good. Your task is to find an integer y ($2 \leq y \leq 10^9$) such that both of the following conditions are satisfied:

- y is good.
- $x \times y$ is good.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of each test case follows.

Each test case contains a single integer x ($1 \leq x < 10^8$). It is guaranteed that x is good.

Output

For each test case, print a single integer y ($2 \leq y \leq 10^9$) such that both the integers y and $x \times y$ are good.

If there are multiple valid answers, output any one of them.

input
4
8
73
299
6767
output
11
4
26
3366

For the first test case, we have $x = 8$; choosing $y = 11$ is valid because both $y = 11$ and $x \times y = 88$ are good.

For the second test case, we have $x = 73$; choosing $y = 4$ is valid because both $y = 4$ and $x \times y = 292$ are good.

C. RemovevomeR

1 second, 256 megabytes

You are given a binary string s consisting only of the characters 0 and 1.

In one operation, you can do the following:

- Choose a substring* of s that is a palindrome† of length at least 2.
- Delete exactly one character from this chosen substring.

The remaining parts of the string are then concatenated to form the new string s .

Find the minimum possible length of the string s that can be achieved after applying this operation any number of times (possibly zero).

*A string a is a substring of a string b if a can be obtained from b by the deletion of several (possibly, zero or all) characters from the beginning and several (possibly, zero or all) characters from the end.

†A string a of length m is said to be palindrome if $a_i = a_{m+1-i}$ for all $1 \leq i \leq m$.

Input

The first line contains a single integer t ($1 \leq t \leq 100$) — the number of test cases. Description of each test case follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 100$) — the length of the binary string s .

The second line of each test case contains a binary string s of length n . It is guaranteed that each character of s is either 0 or 1.

Output

For each test case, print the minimum possible length of the string s that can be achieved after applying the operation any number of times.

input
4
4
0000
3
110
6
110011
6
101100

output
1
2
1
1

In the first test case, the initial string is 0000. We can perform the following sequence of operations:

- Choose the palindromic substring 0000. Delete one 0. The string becomes 000.
- Choose the palindromic substring 000. Delete one 0. The string becomes 00.
- Choose the palindromic substring 00. Delete one 0. The string becomes 0.

The string 0 contains no palindromic substrings of length at least 2, so no further operations can be performed. The minimum possible length is 1.

In the second test case, the initial string is 110.

- Choose the palindromic substring 11. Delete one 1. The string becomes 10.

The string 10 contains no palindromic substrings of length at least 2, so no further operations can be performed. The minimum possible length is 2.

D. An Alternative Way

2 seconds, 256 megabytes

You are given two arrays a and b , each of length n . You are allowed to perform the following operation on array a any number of times (including zero):

- Choose two indices l and r such that $1 \leq l \leq r \leq n$;
- For each index i from l to r (both inclusive),
 - Set $a_i := a_i - 1$ if $i - l$ is odd.
 - Set $a_i := a_i + 1$ if $i - l$ is even.

Determine whether you can make the array a equal to the array b by performing the operation any number of times.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. Description of each test case follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of the arrays a and b .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the elements of the array a .

The third line of each test case contains n integers $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$) — the elements of the array b .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print "YES" if you can make array a equal to array b and "NO" otherwise.

You can output "YES" and "NO" in any case (for example, strings "yEs", "yes" and "Yes" will be recognized as a positive response).

input
7
3
1 2 3
1 2 3
4
1 4 5 2
1 5 4 3
1
9
8
6
6 7 6 7 6 7
7 6 7 6 7 6
9
9 8 7 6 5 4 3 2 1
9 9 8 2 4 4 3 5 3
3
1 1 2
2 1 1
2
1 2
1 1
output
YES
YES
NO
YES
NO
YES
NO

For the first test case, arrays a and b are already equal.

For the second test case, let us choose $l = 2$ and $r = 4$. Now, we update the array a in the following manner:

- For $i = 2$, we have $i - l = 2 - 2 = 0$, which is even. Hence, set $a_2 := a_2 + 1 = 4 + 1 = 5$.
- For $i = 3$, we have $i - l = 3 - 2 = 1$, which is odd. Hence, set $a_3 := a_3 - 1 = 5 - 1 = 4$.
- For $i = 4$, we have $i - l = 4 - 2 = 2$, which is even. Hence, set $a_4 := a_4 + 1 = 2 + 1 = 3$.

Finally, we have array $a = [1, 5, 4, 3]$ and array $b = [1, 5, 4, 3]$.

For the third test case, it can be shown that it is impossible to make array a equal to array b .

E. Fair and Square

3 seconds, 256 megabytes

A tree is an undirected connected graph with no cycles.

You are given a tree having n vertices. Each vertex i has an integer value a_i written on it.

For any two vertices u and v ($u \neq v$), define $p(u, v)$ as the product of the values written on the vertices lying on the unique simple path* from u to v .

An unordered triplet of three distinct vertices $\{u, v, w\}$ is called *good* if and only if: $p(u, v) \cdot p(v, w) \cdot p(w, u)$ is a **perfect square**.

Determine the number of *good* unordered triplets in the given tree.

*A simple path from the vertex u to vertex v is a sequence of distinct vertices $u = x_0, x_1, \dots, x_k = v$ such that there exists an edge between vertices x_{i-1} and x_i for all $1 \leq i \leq k$.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of each test case follows.

Each test case begins with an integer n ($3 \leq n \leq 2 \cdot 10^5$) — the number of vertices.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^6$) — the integer values written on the vertices.

Each of the next $n - 1$ lines contains two integers u, v ($1 \leq u, v \leq n$), denoting an edge of the tree. It is guaranteed that the edges form a tree.

It is guaranteed that the sum of n over all the test cases does not exceed $2 \cdot 10^5$.

Output

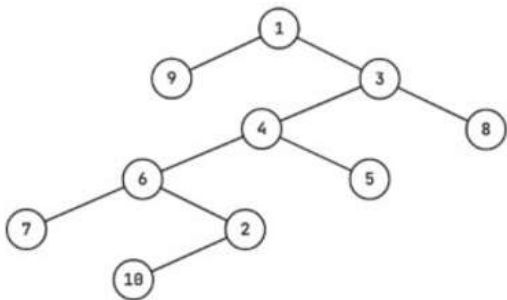
For each test case output the number of good triplets in the tree.

input
4
5
1 1 1 1 1
1 2
2 3
2 4
4 5
10
1 2 3 4 5 6 7 8 9 10
1 3
2 6
6 7
5 4
8 3
3 4
4 6
9 1
10 2
6
12 6 3 18 9 2
3 4
4 5
2 6
6 1
4 2
8
3 16 9 1 8 16 4 9
2 1
3 1
4 3
3 5
6 3
4 7
8 1
output
10
48
0
40

For the first test case, all the unordered triplets of three distinct vertices are good:

- 1. {1, 2, 3}
- 2. {1, 2, 4}
- 3. {1, 2, 5}
- 4. {1, 3, 4}
- 5. {1, 3, 5}
- 6. {1, 4, 5}
- 7. {2, 3, 4}
- 8. {2, 3, 5}
- 9. {2, 4, 5}
- 10. {3, 4, 5}

For the second test case, {2, 5, 8} is a good triplet.



F. A Bit Odd

2 seconds, 256 megabytes

Alice and Bob have got a binary* string s of length n . They have decided to play a game on it, taking turns alternately, with Alice moving first.

In each move, the player must select a **subsequence**[†] which has an **odd** number of inversions[‡] and delete it. The player who cannot make a move loses.

Determine who wins the game, assuming both players play optimally.

*A binary string is a string that consists only of the characters 0 and 1.

[†]A sequence a is a subsequence of a string b if a can be obtained from b by the deletion of several (possibly zero or all) characters.

[‡]An inversion in a binary string s is a pair of indices (i, j) such that $i < j$ and $s_i = 1$ and $s_j = 0$.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. Description of each test case follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of the binary string s .

The second line of each test case contains a binary string s of length n . It is guaranteed that each character of s is either 0 or 1.

It is guaranteed that the sum of n over all the test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print **Alice** if Alice wins the game and **Bob** otherwise.

input
3
5
10101
4
0100
6
011001
output
Alice
Alice
Bob

For the first test case, Alice can choose the entire string as it has an odd number of inversions. Now, Bob is left with an empty string, and he cannot make a move. Thus, Alice wins.

For the second test case, Alice can choose the subsequence formed by the characters at indices 1, 2, and 4, i.e., 010. Bob is then left with the character at index 3, namely 0, which has 0 inversions (an even number). Therefore, Bob cannot choose a subsequence with an odd number of inversions, so Alice wins.

For the third test case, it can be shown that Bob can guarantee a win irrespective of Alice's first move.

G. Summmon

4 seconds, 256 megabytes

For any array b of length m , define $f(b)$ as the **minimum** possible value of $\max(b) - \min(b)$ that can be achieved by performing the following operation any number of times:

- Choose any index $1 \leq i < m$, and do exactly one of the following:
 - 1. Set $b_{i+1} := b_{i+1} + b_i$,
 - 2. Set $b_{i+1} := b_{i+1} - b_i$.

You are given an array a of length n . Your task is to compute the sum of f over all the subarrays* of a . More formally, determine the value of

$$\sum_{1 \leq l \leq r \leq n} f([a_l, a_{l+1}, \dots, a_r]).$$

*An array b is a subarray of an array a if b can be obtained from a by deletion of several (possibly, zero or all) elements from the beginning and several (possibly, zero or all) elements from the end. In particular, an array is a subarray of itself.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases. Description of each test case follows.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of the array a .

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the elements of the array a .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer — the value of

$$\sum_{1 \leq l \leq r \leq n} f([a_l, a_{l+1}, \dots, a_r]).$$

input
5
3
6 4 8
4
1 2 3 4
9
9 9 8 2 4 4 3 5 3
6
18 12 24 9 6 36
6
36 24 18 12 9 6
output
4
3
39
72
111

For the first test case, let us look at all subarrays.

- For the single-element subarrays $[6]$, $[4]$, and $[8]$, no operation can change anything, so each of them contributes 0.
- For $[6, 4]$, the first element is fixed as 6. The second element can only be changed to $4 + 6 = 10$ or $4 - 6 = -2$, so the gap would become 4 or 8. Hence, doing no operation is best, and the contribution is 2.
- For $[4, 8]$, we choose $i = 1$ and apply $b_2 := b_2 - b_1$. Then the subarray becomes $[4, 4]$, so its contribution is 0.
- For $[6, 4, 8]$, we keep the first two elements as they are and choose $i = 2$. Then we apply $b_3 := b_3 - b_2$, so 8 becomes 4. The array becomes $[6, 4, 4]$, and therefore $\max(b) - \min(b) = 6 - 4 = 2$. It can be shown that this is the optimal value.

Therefore, the answer for the first test case is

$$0 + 0 + 0 + 2 + 0 + 2 = 4.$$

For the second test case, every subarray of length 1 contributes 0.

Among the remaining subarrays, only $[2, 3]$, $[3, 4]$, and $[2, 3, 4]$ have a non-zero contribution, each contributing 1. Therefore, the answer is $1 + 1 + 1 = 3$.